

实验三题解

链表合并

思路

`merge()` 函数：通过比较两个输入链表的节点值，依次选择较小（或按特定顺序）的节点插入到新链表的尾部，直到两个输入链表都遍历完毕，最后返回合并后的新链表。

注意在遍历2个输入链表时的四种情况（其中 `p1` 和 `p2` 分别为用于遍历列表的指针）：

- `p1 == NULL && p2 != NULL`：将 `p2` 及其后续节点全部插入到新链表。
- `p1 != NULL && p2 == NULL`：将 `p1` 及其后续节点全部插入到新链表。
- `p1 != NULL && p2 != NULL`：比较 `p1` 和 `p2` 的值，将较小值的节点插入到新链表，并移动相应的指针。
- `p1 == NULL && p2 == NULL`：两个链表都已遍历完，跳出循环。

代码

```
node* createList()
{
    node* head = NULL;
    node* p1, * p2;
    p1 = new node;
    p2 = p1;
    cin >> p1->val;
    while (p1->val != -1)
    {
        if (head == NULL)
            head = p1;
        else p2->next = p1;
        p2 = p1;
        p1 = new struct node;
        cin >> p1->val;
    }
    p2->next = NULL;
    delete p1;
    return head;
}

node* merge(node* head1, node* head2)
{

```

```

node *p1 = head1;
node *p2 = head2;
node *head3 = new node; //新链表的头结点，不存数据，和头指针的意义不同
head3->next = NULL;
node* tail = head3; // 尾部插点法构造新链表

```

// 分四种情况讨论

```

while (true)
{
    if (p1 == NULL && p2 != NULL)
    {
        while (true)
        {
            tail->next = p2; // 在新链表尾部插入结点
            tail = tail->next; // 更新链表尾部指针
            p2 = p2->next;
            if (p2 == NULL)
                break;
        }
    }
    else if (p1 != NULL && p2 == NULL)
    {
        while (true)
        {
            tail->next = p1; // 在新链表尾部插入结点
            tail = tail->next; // 更新链表尾部指针
            p1 = p1->next;
            if (p1 == NULL)
                break;
        }
    }
    else if (p1 != NULL && p2 != NULL)
    {
        if (p1->val < p2->val)
        {
            tail->next = p1; // 在新链表尾部插入结点
            tail = tail->next;
            p1 = p1->next; // 向后移动指针p1
        }
        else
        {
            tail->next = p2; // 在新链表尾部插入结点
            tail = tail->next;
            p2 = p2->next; // 向后移动指针p2
        }
    }
    else if (p1 == NULL && p2 == NULL)

```

```

        break;
    }

    tail->next = NULL; // 新链表的尾部置为NULL
    node* tmp = head3->next; // 新链表中仍然使用头指针，不使用头结点
    delete head3; // 删除头结点
    head3 = tmp;
    return head3;
}

```

倒数第n个节点

思路

使用双指针（快指针和慢指针）遍历链表，快指针先向前移动k步，然后快慢指针同时移动，当快指针到达链表末尾时，慢指针指向的节点即为从链表尾部往前数的第k个节点。

具体步骤如下：

1. `fast` 和 `slow` 指针都初始化为指向链表的头节点 `head`。
2. 使用一个循环，让 `fast` 指针先移动k步。如果在这个过程中 `fast` 指针到达链表尾部（即 `fast == NULL`），说明k大于链表长度，直接返回 `NULL`。
3. 当 `fast` 指针移动k步后，开始一个新的循环，让 `fast` 和 `slow` 指针同时移动，直到 `fast` 指针到达链表尾部。
4. 当 `fast` 指针到达链表尾部时，`slow` 指针正好指向从尾部开始的第k个节点，返回 `slow` 指针。

代码

```

ListNode* getKthFromEnd(ListNode* head, int k) {
    ListNode* fast = head;
    ListNode* slow = head;

    // 快指针先移动k步
    while (fast && k >= 0) {
        fast = fast->next;
        k--;
    }

    // 如果k仍然大于0，说明链表长度小于k，返回NULL

```

```

    if(k >= 0) return NULL;
    // 快慢指针同时移动，直到快指针到达链表尾部
    while (fast) {
        fast = fast->next;
        slow = slow->next;
    }
    return slow;
}

```

后半链表

思路

首先遍历链表计算其长度 n ，然后再次遍历链表，当遍历到链表的中间位置（即第 $n/2$ 个点）时停止，并返回该中间节点。

这种方法基于两次遍历链表，第一次获取长度，第二次定位中间节点。

代码

```

ListNode* middleNode(ListNode* head) {
    int n = 0;
    ListNode* cur = head;
    while (cur != nullptr) {
        ++n;
        cur = cur->next;
    }
    int k = 0;
    cur = head;
    while (k < n / 2) {
        ++k;
        cur = cur->next;
    }
    return cur;
}

```

合并子链表

思路

首先遍历链表，将遇到的非零值累加起来，当遇到值为0的节点时，将累加的总和替换该0节点的值，并继续遍历。为了避免额外的节点分配，直接在原链表上进行修改，通过调整节点间的链接和值来完成合并。最终，函数返回修改后的链表头节点的下一个节点（因为头节点是哑节点，不存储有效值）。

在遍历过程中，使用两个指针 `prev` 和 `curr` 分别追踪当前处理节点的前一个节点和当前节点，以及一个变量 `total` 来记录非零值的累加和。

代码

```
ListNode* mergeNodes(ListNode* head) {
    ListNode* prev = head; // prev 用于追踪合并后的链表尾部
    ListNode* curr = head->next; // 从第一个有效节点开始遍历
    int total = 0;

    while (curr) {
        if (curr->val == 0) {
            // 遇到 0，将当前累计值存入 prev 的下一个节点（即当前 0 节点）
            prev->next = curr; // 链接到合并后的链表
            curr->val = total; // 修改当前 0 节点的值 of 累计和
            prev = curr;      // 移动 prev 到新尾部
            total = 0;        // 重置累计值
        } else {
            total += curr->val; // 累计非零值
        }
        curr = curr->next;
    }
    prev->next = nullptr; // 断开末尾多余的节点

    return head->next; // 返回合并后的链表头节点
}
```

链表重排

思路

首先将空链表作为特殊情况处理，直接返回。

对于非空链表，它使用两个指针 `odd` 和 `even` 分别追踪奇数位置和偶数位置的节点。开始时，`odd` 指向头节点，`even` 指向头节点的下一个节点（即第一个偶数位置的节点）。

然后，通过遍历链表，`odd` 和 `even` 分别跳过它们各自的下一个节点（即 `odd` 指向 `even->next`，然后 `even` 指向 `odd->next` 的下一个节点），直到遍历完所有节点。

最后，将奇数链表的末尾连接到偶数链表的头部，形成交错排列的链表，并返回原始头节点。

代码

```
ListNode* oddEvenList(ListNode* head) {
    if (head == nullptr) {
        return head;
    }
    ListNode* evenHead = head->next;
    ListNode* odd = head;
    ListNode* even = evenHead;
    while (even != nullptr && even->next != nullptr) {
        odd->next = even->next;
        odd = odd->next;
        even->next = odd->next;
        even = even->next;
    }
    odd->next = evenHead;
    return head;
}
```

是否回文

思路

把链表的数字用数组存起来，然后遍历判断是否回文。

代码

```
int isPalindrome(ListNode* head) {
    //这个函数需要你补充完整
    if(head==NULL) return -1;
    int ve[1000], cnt = 0;
    ListNode *cur = head;
    while(cur!=NULL){
        ve[cnt++] = cur->val;
        cur=cur->next;
    }
    bool ok=1;
    for(int i=0,j=cnt-1; i<=j; ++i,--j){
        if(ve[i]!=ve[j]) ok=0;
    }
}
```

```
    return ok;
}
```

链表加法

思路

还是先用两个数组存下链表的值，然后我们模拟加法的过程。模拟加法有多种思路，这里提供一种参考：我们从数字的最低位开始，使用一个变量 t 记录当前位的值，每一次循环，如果 a 还有，那么 $t+=a[i]$ ，如果 b 还有，那么 $t+=b[i]$ 。使用数组 c 存储加法后的结果， $c[i]=t\%10$ ， $t/=10$ 。循环完了以后还要注意 t 如果不为 0 ，那么将 t 存入 c 数组。（这种思路看不懂的同学，大家可以上网搜索如何使用数组模拟加法）。

代码

```
ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
    //你需要补充的代码
    int a[200], b[200], c[200];
    int p1=0, p2=0;
    ListNode *cur=l1;
    //获取链表l1的值
    while(cur!=NULL) {
        a[p1++]=cur->val;
        cur=cur->next;
    }
    //获取链表l2的值
    cur=l2;
    while(cur!=NULL) {
        b[p2++]=cur->val;
        cur=cur->next;
    }
    //数组模拟加法
    int p3=0, t=0;
    for(int i=p1-1, j=p2-1; i>=0 || j>=0; --i, --j) {
        if(i>=0) t+=a[i];
        if(j>=0) t+=b[j];
        c[p3++]=t%10;
        t/=10;
    }
    if(t) c[p3++]=t;
    //创建新链表
    ListNode *head, *tail, *pre;
    if(!p3) return NULL;
```

```
else {  
    // 尾插法创建链表  
    tail = new ListNode;  
    tail->val=c[0];  
    tail->next=NULL;  
    head=pre=tail;  
    int idx=1;  
    while(idx<p3) {  
        head = new ListNode;  
        head->val = c[idx++];  
        head->next=pre;  
        pre=head;  
    }  
    return head;  
}  
}
```